

PayForMe Partner Bank API

Version: 1.0 (draft)

Base URL: `https://<your-domain>/api/bank/v1`

Currency: GHS

Audience: Licensed partner banks integrating with the PayForMe platform

1. Overview

PayForMe exposes a **Partner Bank API** so banks can:

1. **Credit user wallets** when a customer deposits into the platform collection account
2. **Debit user wallets** when the bank has paid out a withdrawal to the customer's linked account
3. **Charge linked bank accounts** for scheduled repayments (mandates / installments)
4. **Report asynchronous status** (`PENDING` , `PROCESSING` , `COMPLETED` , `FAILED`) so the platform UI and ledger stay in sync

This is the **inverse** of MoMo integration: **banks call PayForMe**, not the other way around (except mandate registration, which may still require bank-side endpoints — see §8).

Integration goals

Goal	How it works
User adds bank account	User verifies account in app (Paystack resolve + allowlist). Bank receives <code>bankAccountId</code> only through PayForMe-initiated flows.
User deposits	Customer transfers to one platform collection account with a unique reference → bank notifies PayForMe → wallet credited
User withdraws	User confirms in app (OTP + 2FA) → PayForMe instructs bank OR bank debits wallet after payout
Repayment schedule	PayForMe creates charge against mandate/installment → bank debits customer account → bank reports result
UI status	Every transaction has a lifecycle; users see status in Wallet, Repayments, and Mandates screens

2. Authentication

All Partner Bank API requests must include:

```
x-bank-api-key: <BANK_API_KEY>
Content-Type: application/json
```

Environment variable (PayForMe)	Purpose
<code>BANK_API_KEY</code>	Shared secret. Banks send this in <code>x-bank-api-key</code> .

BANK_API_URL	(Optional, outbound) Base URL if PayForMe also calls your bank for mandates/debits
--------------	--

Security requirements

- TLS 1.2+ only
- Rotate keys quarterly or on compromise
- Whitelist PayForMe egress IPs in production (provided during onboarding)
- Never log full account numbers in plain text

Errors

HTTP	Code	Meaning
401	BANK_API_UNAUTHORIZED	Missing or invalid API key
503	BANK_API_DISABLED	Partner API not enabled on this environment

3. Response envelope

All responses use the same JSON envelope:

Success

```
{
  "success": true,
  "message": "Request completed successfully.",
  "data": { },
  "errors": null,
  "requestId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Error

```
{
  "success": false,
  "message": "Verified bank account required",
  "data": null,
  "errors": [
    {
      "code": "VALIDATION_ERROR",
      "message": "Verified bank account required"
    }
  ],
  "requestId": "550e8400-e29b-41d4-a716-446655440000"
}
```

Include `requestId` when contacting PayForMe support.

4. Idempotency

Every **mutating** request must include a bank-generated `reference` (minimum 6 characters, globally unique per transaction type).

Rule	Behavior
First request with new reference	Process normally → 201 Created
Repeat request with same reference and COMPLETED status	Return existing result → 200 OK, alreadyProcessed: true
Repeat with different amount/user	409 Conflict — DUPLICATE_REFERENCE

Store `reference` in your core banking system before calling PayForMe.

5. Transaction status model

PayForMe uses a unified status enum for wallet and partner transactions:

Status	Meaning	UI
PENDING	Instruction received, not yet settled	"Processing..."
PROCESSING	Bank is executing (e.g. GhIPSS in flight)	"Processing..."
COMPLETED	Funds confirmed; ledger updated	"Successful"
FAILED	Definitive failure	"Failed" + reason
CANCELLED	Voided before settlement	"Cancelled"

Handshake pattern (recommended for production)

1. Bank receives customer action (deposit, withdrawal, debit)
2. Bank calls PayForMe with status PENDING or PROCESSING (optional pre-ack)
3. Bank settles on core banking / GhIPSS
4. Bank calls PayForMe status update OR webhook → COMPLETED | FAILED
5. PayForMe updates WalletTransaction, WithdrawalRequest, DeductionEvent
6. User receives in-app + email notification; UI refreshes

Note: Endpoints marked **Live** below currently apply credits/debits **synchronously** as `COMPLETED`. Async status endpoints are **Planned** (§7).

6. Platform collection account

All customer **bank deposits** must settle into **one designated PayForMe collection account** per environment.

Field	Description
bankName	e.g. GCB Bank
bankCode	GhIPSS / Paystack code

accountNumber	Collection account number
accountName	PayForMe Ghana Ltd (example)

Customer deposit flow

1. User opens **Wallet → Deposit via bank** in the app
2. PayForMe shows collection account + **unique deposit reference** (e.g. PFM-DEP-<userId-short>-<random>)
3. Customer transfers exact amount from their linked bank, using that reference in narration
4. Bank core detects incoming transfer → calls `POST /deposits` with `reference` , `amount` , `userId`
5. PayForMe validates amount + reference → credits wallet → notifies user

Planned endpoint for apps: `GET /deposit-instructions` (returns collection account + generated reference).

Admin configuration (planned): `PlatformSettlementAccount` in admin settings.

7. API reference

Legend

- **Live** — implemented in this repository today
- **Planned** — specified for partner onboarding; implement before production go-live

7.1 Health check — Live

`GET /health`

Verify connectivity and API key.

Response `200`

```
{
  "status": "ok",
  "version": "1.0",
  "environment": "sandbox"
}
```

7.2 Credit user wallet (deposit) — Live

`POST /deposits`

Call when a customer deposit to the **platform collection account** is confirmed.

Request

```
{
  "userId": "clx1234567890abcdefghij",
  "amount": 1500.00,
  "reference": "GCB-IN-20260725-0001842",
  "bankCode": "040",
}
```

```
"description": "Wallet top-up via GCB"
}
```

Field	Type	Required	Description
userId	string (cuid)	Yes	PayForMe user ID
amount	number	Yes	Gross amount in GHS (2 decimal places)
reference	string	Yes	Unique bank transaction reference (min 6 chars)
bankCode	string	No	Originating bank code
description	string	No	Narration for audit

Response 201

```
{
  "success": true,
  "data": {
    "alreadyProcessed": false,
    "transaction": {
      "id": "clxtx9876543210fedcba",
      "walletId": "clxwallet111",
      "type": "DEPOSIT",
      "status": "COMPLETED",
      "amount": "1500.00",
      "fee": "30.00",
      "commission": "0.00",
      "netAmount": "1470.00",
      "reference": "GCB-IN-20260725-0001842",
      "description": "Bank deposit – GCB-IN-20260725-0001842",
      "createdAt": "2026-07-25T10:15:00.000Z",
      "updatedAt": "2026-07-25T10:15:00.000Z"
    }
  }
}
```

Side effects

- Credits user wallet (net of platform fees)
- Sends PAYMENT_SUCCESSFUL in-app + email notification
- Visible on user **Wallet** screen

Errors

HTTP	Condition
400	Invalid payload
404	userId not found
400	User role cannot receive deposits

Implementation: `app/api/bank/v1/deposits/route.ts` , `lib/services/payment/bank-api.service.ts`

7.3 Debit user wallet (withdrawal settlement) — Live

`POST /withdrawals`

Call after the bank has **successfully paid** the customer to their saved PayForMe-linked account. This debits the user's platform wallet.

Prerequisites

- User has a **verified** `bankAccountId` in PayForMe
- User completed in-app withdrawal authorization (OTP + 2FA) — **Planned:** `link via withdrawalRequestId`
- Sufficient wallet balance

Request

```
{
  "userId": "clx1234567890abcdefghij",
  "bankAccountId": "clxbank222",
  "amount": 500.00,
  "reference": "GCB-OUT-20260725-0000911",
  "description": "Wallet withdrawal to GCB *****4521"
}
```

Field	Type	Required	Description
userId	string (cuid)	Yes	PayForMe user ID
bankAccountId	string (cuid)	Yes	Saved verified bank account
amount	number	Yes	Payout amount in GHS
reference	string	Yes	Unique bank payout reference
description	string	No	Audit narration

Response 201

```
{
  "success": true,
  "data": {
    "alreadyProcessed": false,
    "transaction": {
      "id": "clxtx555",
      "type": "WITHDRAWAL",
      "status": "COMPLETED",
      "amount": "500.00",
      "reference": "WDR-ABC123",
      "netAmount": "500.00"
    }
  }
}
```

```
}  
}
```

Side effects

- Debits wallet
- Creates `WithdrawalRequest` with status `COMPLETED`
- Sends `WITHDRAWAL_COMPLETED` notification

Errors

HTTP	Condition
400	Unverified bank account
400	Insufficient balance
400	User role cannot withdraw

Implementation: `app/api/bank/v1/withdrawals/route.ts`

7.4 Initiate withdrawal (bank executes payout) — Live

`POST /withdrawals/initiate`

PayForMe calls this pattern when the user confirms withdrawal in the app. The bank receives a payout instruction and returns `PROCESSING`.

Request (from PayForMe → bank) — documented for bilateral contract

```
{  
  "withdrawalRequestId": "clxwd333",  
  "userId": "clx123",  
  "bankAccountId": "clxbank222",  
  "amount": 500.00,  
  "reference": "PFM-WDR-20260725-88",  
  "accountNumber": "1234567890",  
  "bankCode": "040",  
  "accountName": "JOHN DOE"  
}
```

Response

```
{  
  "status": "PROCESSING",  
  "bankReference": "GCB-OUT-20260725-0000911",  
  "estimatedSettlement": "2026-07-25T10:30:00Z"  
}
```

Bank later calls `PATCH /withdrawals/{withdrawalRequestId}` or webhook (§9) with final status.

7.5 Update withdrawal status — Live

PATCH /withdrawals/{withdrawalRequestId}

```
{
  "status": "COMPLETED",
  "reference": "GCB-OUT-20260725-0000911",
  "completedAt": "2026-07-25T10:28:00Z"
}
```

Or on failure:

```
{
  "status": "FAILED",
  "failureCode": "INSUFFICIENT_FUNDS",
  "failureMessage": "Customer account could not be credited"
}
```

PayForMe will reverse wallet debit on `FAILED` if funds were reserved.

7.6 Charge bank account (repayment / invoice) — Live

POST /charges

Debit a customer's **linked bank account** for a scheduled repayment or invoice. Used for financing installments and recurring mandates.

Request

```
{
  "reference": "GCB-DD-20260725-0044",
  "userId": "clx123",
  "bankAccountId": "clxbank222",
  "amount": 850.00,
  "currency": "GHS",
  "chargeType": "INSTALLMENT",
  "installmentId": "clxinst444",
  "mandateId": "clxmand555",
  "description": "Rent installment #3 – July 2026"
}
```

Field	Type	Required	Description
chargeType	enum	Yes	INSTALLMENT, INVOICE, MANDATE
installmentId	string	Conditional	Required for INSTALLMENT
mandateId	string	Conditional	Required when debiting via mandate
invoiceId	string	Conditional	Required for INVOICE (future)

Response `202 Accepted` (async debit)


```
{
  "chargeId": "clxcharge666",
  "status": "PROCESSING",
  "reference": "GCB-DD-20260725-0044"
}
```

Status updates: webhook `charge.completed` / `charge.failed` (\$9) or `GET /charges/{reference}` .

PayForMe internal mapping

- Updates `DeductionEvent.status`
- Updates `Installment.amountPaid` and `Installment.status`
- Notifies user on success/failure
- Visible on **Repayments** and **Mandates** UI

Today, installment debits are initiated **outbound** via `POST {BANK_API_URL}/debit` from PayForMe cron.
The **inbound** `POST /charges` model above is the target for banks that prefer to own the debit initiation.

7.7 Get transaction status — Live

`GET /transactions/{reference}`

Reconciliation and support lookup.

Response

```
{
  "reference": "GCB-IN-20260725-0001842",
  "type": "DEPOSIT",
  "status": "COMPLETED",
  "amount": "1500.00",
  "userId": "clx123",
  "createdAt": "2026-07-25T10:15:00Z",
  "completedAt": "2026-07-25T10:15:01Z"
}
```

7.8 User lookup by account — Live

`GET /users/lookup?accountNumber=1234567890&bankCode=040`

Map an incoming transfer to `userId` when the deposit reference is missing or malformed. Returns masked identity for bank validation.

Response

```
{
  "userId": "clx123",
  "fullName": "John Doe",
  "email": "j***@example.com",
  "defaultBankAccountId": "clxbank222"
}
```

7.9 Mandate status callback — Live

POST /mandates/callback

Notify PayForMe when a direct-debit mandate is approved, rejected, or revoked.

```
{
  "mandateId": "clxmand555",
  "providerReference": "BANK-MAND-9988",
  "status": "ACTIVE",
  "activatedAt": "2026-07-20T08:00:00Z"
}
```

Status	PayForMe MandateStatus
ACTIVE	ACTIVE
REJECTED	REJECTED
REVOKED	REVOKED
EXPIRED	EXPIRED

8. Mandates and repayment schedules

8.1 User journey

1. User completes KYC and adds verified bank account (BANK type, allowlisted bank)
2. User creates financing / rent plan → **Repayment schedule** (Installment rows with dueDate , amount)
3. User signs **direct debit mandate** linked to bankAccountId
4. On each due date, PayForMe triggers a charge against the mandate
5. Bank debits customer → reports result → installment marked paid → UI updated

8.2 Data objects (PayForMe)

Object	Purpose
BankAccount	Linked customer account (id, bankCode, accountNumber, isVerified)
Mandate	Direct debit authorization (status, providerReference)
Installment	Scheduled payment line (dueDate, amount, amountPaid, status)
DeductionEvent	Single debit attempt (status, providerReference, retries)
WalletTransaction	Ledger entry for deposits/withdrawals

8.3 Outbound bank API (PayForMe → bank) — optional

If your bank exposes REST endpoints, PayForMe can call:

Method	Path	Purpose
--------	------	---------

POST	{BANK_API_URL}/mandates	Register mandate
GET	{BANK_API_URL}/mandates/{ref}	Poll mandate status
POST	{BANK_API_URL}/debit	Execute direct debit

Debit request (PayForMe → bank)

```
{
  "mandateId": "clxmand555",
  "amount": "850.00",
  "currency": "GHS",
  "description": "Installment #3",
  "reference": "clxdeduction777",
  "timestamp": "2026-07-25T06:00:00.000Z"
}
```

Debit response

```
{
  "status": "COMPLETED",
  "transactionId": "BANK-TX-12345"
}
```

Configure `BANK_API_URL` and use the same `BANK_API_KEY` as Bearer token for outbound calls.

9. Webhooks (bank → PayForMe) — Live

For async GhIPSS settlement, banks should POST events to:

URL	Event
POST /api/webhooks/bank	Unified webhook (event field in body)

Headers

```
x-bank-api-key: <BANK_API_KEY>
x-bank-signature: sha256=<HMAC-SHA256(raw body, webhook secret)>
x-bank-event-id: <unique-event-id>
```

Example: deposit completed

```
{
  "event": "deposit.completed",
  "reference": "GCB-IN-20260725-0001842",
  "userId": "clx123",
  "amount": "1500.00",
  "status": "COMPLETED",
}
```

```
"completedAt": "2026-07-25T10:15:00Z"
}
```

PayForMe responds `200` with `{ "received": true }`. Retry with exponential backoff on `5xx`.

10. End-user UI mapping

User action	API / flow	UI location
Add bank account	App: Paystack resolve + POST <code>/api/settings/bank-account</code>	Settings
Deposit via bank	Collection account + POST <code>/deposits</code>	Wallet
Withdraw	OTP/2FA + bank payout + POST <code>/withdrawals</code>	Wallet
View transaction history	WalletTransaction ledger	Wallet
Mandate setup	POST <code>/api/mandates</code>	Mandates
Repayment due	POST <code>/charges</code> or <code>outbound /debit</code>	Repayments
Notifications	In-app + email on status change	Notification bell

Statuses shown to users mirror §5: **Pending, Processing, Successful, Failed.**

11. Fees

Wallet deposits via bank API are subject to platform fees (deducted before credit):

Fee	Env variable	Default
Service fee	SERVICE_FEE_PERCENT	1.5%
Commission	COMMISSION_FEE_PERCENT	2.0%
Processing	PROCESSING_FEE_PERCENT	0.5%

The API response includes `amount` (gross), `fee`, `commission`, and `netAmount` (credited).

Withdrawals and mandate debits: fees per business rules / financing agreement.

12. Sandbox vs production

	Sandbox	Production
Base URL	<code>https://sandbox.payforme.example/api/bank/v1</code>	<code>https://api.payforme.example/api/bank/v</code>
API key	Test key from PayForMe onboarding	Live key
Amounts	Use small test amounts	Real GHS
Idempotency	Required (same as prod)	Required

Outbound debits	Simulated if BANK_API_KEY unset on PayForMe dev	Live GhIPSS
-----------------	---	-------------

PayForMe dev fallback: When `BANK_API_KEY` is not set, partner endpoints return `503 BANK_API_DISABLED`. Mandate debits are simulated locally for QA.

13. Error codes

Code	HTTP	Description
BANK_API_UNAUTHORIZED	401	Invalid API key
BANK_API_DISABLED	503	Partner API not configured
VALIDATION_ERROR	400	Schema / business validation failed
USER_NOT_FOUND	404	Unknown userId
BANK_ACCOUNT_NOT_FOUND	404	Unknown or unverified bankAccountId
INSUFFICIENT_BALANCE	400	Wallet debit would overdraw
DUPLICATE_REFERENCE	409	Same reference, conflicting payload
INSTALLMENT_NOT_DUE	400	Charge before due date
MANDATE_INACTIVE	400	Mandate not ACTIVE

14. Integration checklist

Bank engineering

- ☐ Obtain sandbox `BANK_API_KEY` and collection account details
- ☐ Implement deposit detection → `POST /deposits`
- ☐ Implement payout confirmation → `POST /withdrawals`
- ☐ Implement direct debit → `POST /charges` + webhooks
- ☐ Implement `GET /transactions/{reference}` reconciliation job
- ☐ Sign webhook HMAC verification
- ☐ Load-test idempotent retries

PayForMe platform

- ☒ `POST /deposits` — credit wallet (sync + async status)
- ☒ `POST /withdrawals` — debit wallet after payout
- ☒ `GET /deposit-instructions` — user wallet UI (`POST /api/payments/bank-deposit-instructions`)
- ☒ `POST /withdrawals/initiate` + `PATCH /withdrawals/{id}` async status
- ☒ `POST /charges` — installment / mandate debits
- ☒ `POST /api/webhooks/bank` unified webhook handler
- ☒ `PlatformSettlementAccount` admin config (`/admin/settings`)

- ☒ Partner `reference` stored on `BankPartnerTransaction` and withdrawals
- ☒ Link `WithdrawalRequest` ↔ partner flow for bank payouts

Compliance

- ☐ PCI / data handling review for account numbers
- ☐ GhIPSS participant agreement
- ☐ Audit log retention (PayForMe `FILE_ACCESS` + transaction metadata)

15. Support

Item	Detail
Technical contact	integrations@payforme.example
Include in tickets	<code>requestId</code> , <code>reference</code> , <code>timestamp</code> , <code>endpoint</code>
Status page	<code>status.payforme.example</code> (planned)

16. Changelog

Version	Date	Changes
1.0-draft	2026-07-25	Initial partner API specification; documents live <code>/deposits</code> and <code>/withdrawals</code> plus planned <code>async</code> , <code>charges</code> , and <code>webhooks</code>

Related documentation

- [RentVest API \(app & admin\)](#)
- Environment variables: `.env.example` (`BANK_API_KEY` , `BANK_API_URL`)
- MoMo collections: separate integration (not covered here)